

1. Theory: Difference Between Top-Down and Bottom-Up Parsing

Top-Down Parsing

Top-down parsing is a method that constructs the parse tree starting from the top, which represents the starting symbol of the grammar, and works its way down to the leaves that represent the terminal symbols. This method primarily utilizes predictive parsing techniques, where a set of production rules is employed to anticipate the next symbols in the input string.

Characteristics:

- **Approach:** Predictive; the parser predicts which production to apply based on the current input symbol.
- **Common Techniques:** Recursive descent, LL parsers.
- **Advantages:**
 - More intuitive and easier to implement, especially for educational purposes.
 - Facilitates modification for diverse grammars.
- **Disadvantages:**
 - Limited to a subset of grammars, notably LL(k) grammars, which can restrict its applicability.
 - May require backtracking for grammars that are ambiguous.

Bottom-Up Parsing

In contrast, bottom-up parsing constructs the parse tree starting from the leaves and working upwards to the root. This method seeks to reduce the input string by applying production rules in reverse, ultimately leading to the starting symbol.

Characteristics:

- **Approach:** Reductive; the parser reduces segments of the input string until it reaches the start symbol.
- **Common Techniques:** Shift-reduce parsers, LR parsers.
- **Advantages:**
 - Capable of handling a broader class of grammars, including all context-free grammars (CFG).
 - More robust in managing ambiguity without requiring backtracking.
- **Disadvantages:**
 - Typically more complex to implement and understand.
 - The underlying algorithms tend to be more challenging to optimize.

Conclusion

The choice between top-down and bottom-up parsing methods largely depends on the specific requirements of the parsing task, including the syntax of the input language and the complexity of the grammar involved.

C++ Program to Check for Balanced Parentheses

```
cpp
#include <iostream>
#include <stack>
#include <string>

bool isBalanced(const std::string &expression) {
    std::stack<char> s;

    for (char ch : expression) {
        if (ch == '(') {
            s.push(ch);
        } else if (ch == ')') {
            // If the stack is empty or top is not '('
            if (s.empty() || s.top() != '(') {
                return false;
            }
            s.pop();
        }
    }

    // At the end, the stack should be empty if balanced
    return s.empty();
}

int main() {
    std::string expression;
    std::cout << "Enter an expression with parentheses: ";
    std::cin >> expression;

    if (isBalanced(expression)) {
        std::cout << "The parentheses are balanced." << std::endl;
    } else {
        std::cout << "The parentheses are not balanced." << std::endl;
    }

    return 0;
}
```

Explanation

- The program uses a stack data structure to track opening parentheses.
- It iterates through each character in the input string and manages the stack accordingly.
- After processing the string, if the stack is empty, it indicates that the parentheses are well-balanced; otherwise, they are not.

3. Problem-Solving: Parse Tree for the Grammar

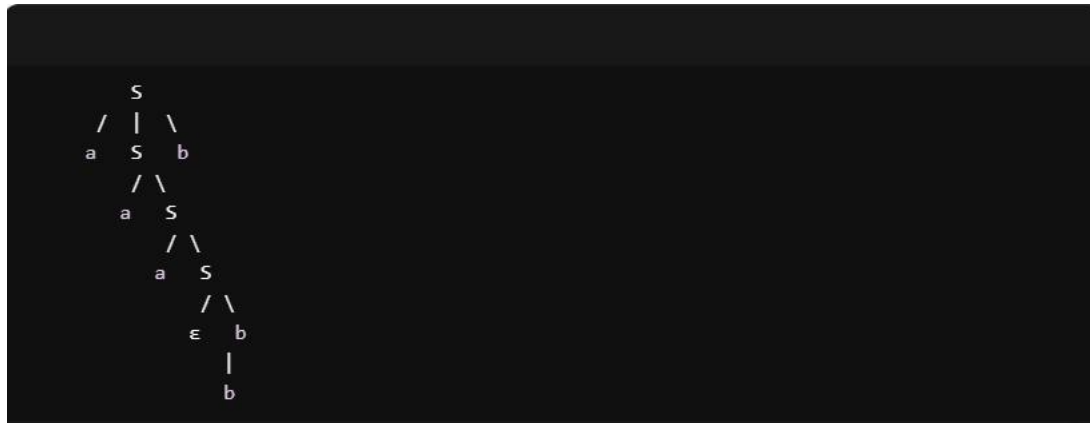
Given the grammar:

- $S \rightarrow aSb \mid \varepsilon$

To derive the string "**aaabbb**":

1. $S \rightarrow aSb$
2. $S \rightarrow aSb$
3. $S \rightarrow aSbb$
4. $S \rightarrow \varepsilon$

Parse Tree Representation



Explanation

- The parse tree illustrates the derivation of "aaabbb" using the provided grammar.
- Each branch represents a production rule application, leading to the final string via accumulated symbols.